

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN ĐA NGÀNH - CÔNG NGHỆ PHẦN MỀM

“Yolo Home”

Instructor(s): ThS. Nguyễn Hữu Hiếu

Class/Group Name: Group 18

Students: Đỗ Phú Khang - 2311409
Hoàng Giữ Tiến Nhất - 2312460
Vũ Thị Kiều Trang - 2313523
Đoàn Minh Hưng - 2311320
Võ Lâm Gia Bảo - 2310273

TP. Hồ Chí Minh - 2026

Mục lục

1	Đặc tả yêu cầu hệ thống	2
1.1	Giới thiệu	2
1.2	Tổng quan hệ thống	2
1.3	Các bên liên quan (Stakeholders)	3
1.4	Phạm vi hệ thống (Scope)	4
1.4.1	In-Scope	4
1.4.2	Out-of-Scope	4
1.5	Danh sách các thiết bị	6
1.6	Phân tích yêu cầu	8
1.6.1	Yêu cầu chức năng	8
1.6.2	Yêu cầu phi chức năng	9
1.7	Use-case Diagram	10
1.8	Use-case Scenarios	13
1.8.1	Module 1: Trực quan hóa dữ liệu (Data Visualization)	13
1.8.2	Module 2: Giám sát và tự động xử lý dữ liệu (Monitoring & Automation)	16
1.8.3	Module 3: Điều khiển thiết bị (Device Control)	18
1.8.4	Module 4: Ghi nhận hoạt động	21
1.8.5	Module 5: Ứng dụng Web	25
1.9	Phân tích nghiệp vụ hệ thống	30
2	Thiết kế hệ thống	33
2.1	Lựa chọn mẫu thiết kế	33
2.1.1	Repository Pattern	33
2.1.2	Dependency Injection Pattern	35
2.1.3	Observer Pattern	37
2.2	Sơ đồ thực thể liên kết mở rộng(EERD)	40
2.3	Sơ đồ ánh xạ thực thể sang mô hình quan hệ	43

1 Đặc tả yêu cầu hệ thống

1.1 Giới thiệu

Trong những năm gần đây, sự phát triển mạnh mẽ của Internet of Things (IoT) đã mở ra nhiều ứng dụng quan trọng trong đời sống, đặc biệt là trong lĩnh vực nhà thông minh (Smart Home). Các hệ thống này cho phép người dùng giám sát và điều khiển môi trường sống từ xa, đồng thời tích hợp các cơ chế tự động nhằm nâng cao mức độ tiện nghi và an toàn.

Dự án Yolo:Home được xây dựng với mục tiêu thiết kế và triển khai một hệ thống nhà thông minh hoàn chỉnh, kết nối giữa phần cứng và phần mềm thông qua nền tảng truyền thông thời gian thực. Hệ thống cho phép thu thập dữ liệu môi trường như nhiệt độ, độ ẩm và chuyển động, sau đó hiển thị trực quan trên giao diện web. Đồng thời, người dùng có thể điều khiển các thiết bị như quạt, đèn cũng như nhận cảnh báo khi phát hiện các điều kiện bất thường.

Mục tiêu của đề tài không chỉ dừng lại ở việc xây dựng một hệ thống hoạt động được, mà còn hướng đến việc thiết kế kiến trúc có tính mở rộng, đảm bảo khả năng tích hợp thêm các chức năng nâng cao trong tương lai như trí tuệ nhân tạo hoặc triển khai trên nền tảng điện toán đám mây.

1.2 Tổng quan hệ thống

Hệ thống Yolo:Home được xây dựng theo kiến trúc phân tầng, bao gồm các thành phần chính: khối cảm biến và thiết bị chấp hành, IoT Gateway, MQTT Broker (Adafruit IO), Backend Server và ứng dụng Web Frontend.

Ở tầng phần cứng, các cảm biến như DHT20 và PIR được kết nối với vi điều khiển Yolo:Bit để thu thập dữ liệu môi trường. Dữ liệu này được truyền đến IoT Gateway thông qua giao tiếp nối tiếp. Gateway đóng vai trò xử lý trung gian, chuẩn hóa dữ liệu và gửi lên MQTT Broker thông qua giao thức MQTT.

Adafruit IO đóng vai trò là trung gian truyền thông, cho phép các thành phần trong hệ thống publish và subscribe dữ liệu theo thời gian thực. Ứng dụng frontend có thể nhận dữ liệu trực

tiếp từ broker để hiển thị realtime, trong khi backend server đồng thời subscribe dữ liệu này để lưu trữ và xử lý.

Backend server chịu trách nhiệm thực hiện các chức năng như lưu trữ dữ liệu vào cơ sở dữ liệu, xử lý logic cảnh báo, và cung cấp các API cho frontend. Người dùng tương tác với toàn bộ hệ thống thông qua giao diện web, nơi cung cấp các chức năng như xem dashboard, điều khiển thiết bị và truy vấn lịch sử.

1.3 Các bên liên quan (Stakeholders)

Trong quá trình phát triển và vận hành hệ thống Yolo:Home, có nhiều bên liên quan (stakeholders) tham gia hoặc bị ảnh hưởng bởi hệ thống. Các stakeholders chính bao gồm:

- **Người dùng cuối (End User)** là đối tượng trực tiếp sử dụng hệ thống thông qua giao diện web. Họ có nhu cầu giám sát môi trường sống, điều khiển thiết bị từ xa và nhận cảnh báo khi có sự cố xảy ra. Đây là stakeholder quan trọng nhất, quyết định các yêu cầu chức năng và trải nghiệm người dùng của hệ thống.
- **Nhóm phát triển (Development Team)** chịu trách nhiệm thiết kế, xây dựng và bảo trì hệ thống. Nhóm này quan tâm đến kiến trúc hệ thống, khả năng mở rộng, hiệu năng và tính ổn định. Các quyết định về công nghệ, thiết kế dữ liệu và triển khai đều do nhóm này thực hiện.
- **Các thiết bị phần cứng (Hardware Devices)**, bao gồm Yolo:Bit, cảm biến DHT20, cảm biến PIR và các thiết bị chấp hành như relay, servo và buzzer, đóng vai trò cung cấp dữ liệu và thực thi hành động trong hệ thống. Mặc dù không phải là con người, nhưng các thiết bị này được xem là stakeholder vì chúng tham gia trực tiếp vào quá trình vận hành hệ thống.
- **Dịch vụ trung gian (Third-party Services)**, cụ thể là Adafruit IO, đóng vai trò là MQTT broker để truyền dữ liệu giữa các thành phần. Dịch vụ này ảnh hưởng đến độ trễ, độ tin cậy và khả năng mở rộng của hệ thống, do đó cũng được xem là một stakeholder quan trọng.

Backend server và cơ sở dữ liệu cũng có thể được xem là stakeholder kỹ thuật, vì chúng chịu trách nhiệm xử lý logic, lưu trữ dữ liệu và cung cấp API cho hệ thống. Các thành phần này

ảnh hưởng trực tiếp đến hiệu năng và độ chính xác của hệ thống.

1.4 Phạm vi hệ thống (Scope)

1.4.1 In-Scope

Trong phạm vi của dự án, hệ thống hỗ trợ đầy đủ các chức năng cốt lõi của một hệ thống nhà thông minh cơ bản. Người dùng có thể đăng nhập vào hệ thống để truy cập các chức năng chính. Sau khi đăng nhập, người dùng có thể quan sát dữ liệu cảm biến như nhiệt độ và độ ẩm theo thời gian thực thông qua dashboard.

Hệ thống cho phép điều khiển thiết bị từ xa, bao gồm bật hoặc tắt các thiết bị như quạt hoặc đèn, cũng như điều khiển các cơ cấu như servo để mở hoặc đóng cửa. Các lệnh điều khiển được gửi thông qua MQTT và được thực thi bởi IoT Gateway.

Ngoài ra, hệ thống còn hỗ trợ lưu trữ dữ liệu cảm biến theo thời gian và cung cấp chức năng xem lịch sử cũng như thống kê dữ liệu. Người dùng có thể theo dõi xu hướng biến đổi của môi trường thông qua các biểu đồ trực quan.

Một chức năng quan trọng khác là cấu hình ngưỡng cảnh báo nhiệt độ. Khi nhiệt độ vượt quá ngưỡng được thiết lập, hệ thống sẽ tự động phát hiện và gửi cảnh báo đến người dùng. Đồng thời, hệ thống cũng ghi lại toàn bộ các hành động điều khiển thiết bị vào nhật ký để phục vụ việc kiểm tra và phân tích.

Hệ thống được xây dựng với kiến trúc hỗ trợ giao tiếp thời gian thực thông qua MQTT, đảm bảo dữ liệu được cập nhật nhanh chóng và chính xác.

1.4.2 Out-of-Scope

Trong phạm vi của đề án hiện tại, một số chức năng nâng cao chưa được triển khai:

- Hệ thống chưa hỗ trợ triển khai trên nền tảng điện toán đám mây quy mô lớn, do đó khả năng mở rộng và chịu tải còn hạn chế. Việc triển khai hiện tại chủ yếu mang tính thử nghiệm và phục vụ mục đích học tập.
- Các cơ chế bảo mật nâng cao như xác thực đa yếu tố (MFA), phân quyền chi tiết

theo vai trò hoặc mã hóa đầu cuối chưa được tích hợp. Hệ thống chỉ sử dụng cơ chế đăng nhập cơ bản.

- Ngoài ra, hệ thống chưa hỗ trợ ứng dụng di động riêng biệt, mà chỉ cung cấp giao diện web. Các tính năng thông minh nâng cao như phân tích dữ liệu bằng trí tuệ nhân tạo, dự đoán hành vi hoặc tự động hóa nâng cao cũng chưa được triển khai.
- Hệ thống cũng chưa tích hợp với các nền tảng nhà thông minh khác như Google Home hoặc Alexa. Việc điều khiển thiết bị vẫn giới hạn trong phạm vi hệ thống nội bộ.
- Cuối cùng, việc quản lý nhiều người dùng với các quyền khác nhau hoặc chia sẻ quyền truy cập giữa các người dùng chưa được hỗ trợ trong phiên bản hiện tại.

1.5 Danh sách các thiết bị

Mạch Yolo Bit

Mạch Yolo:Bit là một vi điều khiển nhỏ gọn được phát triển phục vụ cho giáo dục STEM và các dự án IoT. Thiết bị tích hợp sẵn WiFi, Bluetooth cùng nhiều cảm biến cơ bản như nút nhấn, LED ma trận, giúp người dùng dễ dàng lập trình và triển khai các ứng dụng điều khiển thông minh. Yolo:Bit hỗ trợ lập trình bằng khối lệnh trực quan hoặc Python, phù hợp cho cả người mới bắt đầu và các dự án nâng cao.

Mạch mở rộng chân

Mạch mở rộng chân (Expansion Board) được sử dụng để tăng số lượng chân kết nối của Yolo:Bit, giúp dễ dàng kết nối với nhiều cảm biến và thiết bị ngoại vi cùng lúc. Thiết bị này giúp việc đấu nối trở nên thuận tiện, ổn định và hạn chế sai sót trong quá trình lắp ráp mạch.

Cảm biến Nhiệt độ và Độ ẩm DHT20

Cảm biến DHT20 là thiết bị dùng để đo nhiệt độ và độ ẩm môi trường với độ chính xác cao. Cảm biến sử dụng giao tiếp I2C, cho phép truyền dữ liệu nhanh và ổn định. DHT20 thường được ứng dụng trong các hệ thống giám sát môi trường, nhà thông minh và nông nghiệp thông minh.

Cảm biến ánh sáng

Cảm biến ánh sáng dùng để đo cường độ ánh sáng trong môi trường xung quanh. Dữ liệu thu được có thể dùng để tự động điều chỉnh độ sáng đèn hoặc kích hoạt các thiết bị khi môi trường thiếu sáng. Đây là thành phần quan trọng trong các hệ thống chiếu sáng thông minh.

Cảm biến chuyển động PIR

Cảm biến PIR (Passive Infrared Sensor) được sử dụng để phát hiện chuyển động của con người thông qua bức xạ hồng ngoại. Khi có chuyển động, cảm biến sẽ gửi tín hiệu về vi điều khiển

để kích hoạt các thiết bị như đèn, còi báo động hoặc camera, thường dùng trong hệ thống an ninh.

Remote và mắt nhận hồng ngoại

Remote hồng ngoại là thiết bị điều khiển từ xa, sử dụng tín hiệu hồng ngoại để gửi lệnh. Mắt nhận hồng ngoại có nhiệm vụ thu tín hiệu từ remote và chuyển đổi thành dữ liệu để vi điều khiển xử lý. Bộ đôi này giúp người dùng điều khiển hệ thống một cách tiện lợi mà không cần tiếp xúc trực tiếp.

Động cơ RC Servo

Động cơ RC Servo là loại động cơ có khả năng điều khiển chính xác góc quay. Servo thường được sử dụng trong các ứng dụng cần điều khiển vị trí như cửa tự động, robot hoặc cánh tay cơ khí. Việc điều khiển servo được thực hiện thông qua tín hiệu PWM từ vi điều khiển.

Quạt mini

Quạt mini là thiết bị dùng để tạo luồng không khí, giúp làm mát môi trường hoặc thiết bị. Trong các dự án IoT, quạt thường được điều khiển tự động dựa trên nhiệt độ hoặc bằng điều khiển từ xa.

Đèn RGB

Đèn RGB là loại đèn LED có thể phát ra nhiều màu sắc khác nhau bằng cách kết hợp ba màu cơ bản: đỏ (Red), xanh lá (Green) và xanh dương (Blue). Đèn RGB được sử dụng để hiển thị trạng thái hệ thống, tạo hiệu ứng ánh sáng hoặc trang trí.

Màn hình LCD

Màn hình LCD (Liquid Crystal Display) được sử dụng để hiển thị thông tin như nhiệt độ, độ ẩm, trạng thái thiết bị hoặc thông báo cho người dùng. LCD giúp tăng khả năng tương tác giữa hệ thống và người sử dụng.

1.6 Phân tích yêu cầu

1.6.1 Yêu cầu chức năng

Yêu cầu chức năng tương tác

Các yêu cầu chức năng tương tác mô tả những chức năng mà người dùng có thể trực tiếp thao tác thông qua giao diện hệ thống.

Module 1: Trực quan hóa dữ liệu (Data Visualization)

Hệ thống web cung cấp giao diện hiển thị dữ liệu môi trường theo thời gian thực, bao gồm các thông số như nhiệt độ và trạng thái cảm biến chuyển động. Dữ liệu này được lấy từ server Adafruit thông qua giao thức MQTT và được cập nhật liên tục để người dùng có thể theo dõi tình trạng môi trường một cách chính xác.

Module 3: Điều khiển thiết bị (Device Control)

Hệ thống cho phép người dùng chủ động điều khiển các thiết bị vật lý như cửa, quạt hoặc các thiết bị đầu ra khác thông qua giao diện web. Người dùng có thể thực hiện các thao tác như bật/tắt thiết bị hoặc thay đổi chế độ hoạt động một cách trực tiếp.

Module 5: Quản lý giao diện và xác thực (UI & Authentication Management)

Hệ thống cung cấp chức năng đăng nhập nhằm xác thực người dùng trước khi truy cập. Sau khi đăng nhập thành công, người dùng có thể sử dụng các chức năng như chuyển đổi giữa các màn hình hiển thị dữ liệu và màn hình điều khiển thiết bị, đảm bảo trải nghiệm sử dụng thuận tiện và trực quan.

Yêu cầu chức năng không tương tác

Các yêu cầu chức năng không tương tác là những chức năng được hệ thống tự động thực hiện mà không cần sự can thiệp trực tiếp từ người dùng.

Module 2: Giám sát và tự động xử lý dữ liệu (Monitoring & Automation)

Hệ thống backend liên tục theo dõi dữ liệu được gửi từ các thiết bị thông qua server Adafruit. Khi phát hiện giá trị vượt quá ngưỡng đã được thiết lập, hệ thống sẽ tự động thực hiện các hành động phù hợp, chẳng hạn như gửi thông báo cảnh báo hoặc điều khiển thiết bị nhằm đưa môi trường về trạng thái an toàn.

Module 4: Thu thập và lưu trữ dữ liệu (Data Collection & Storage)

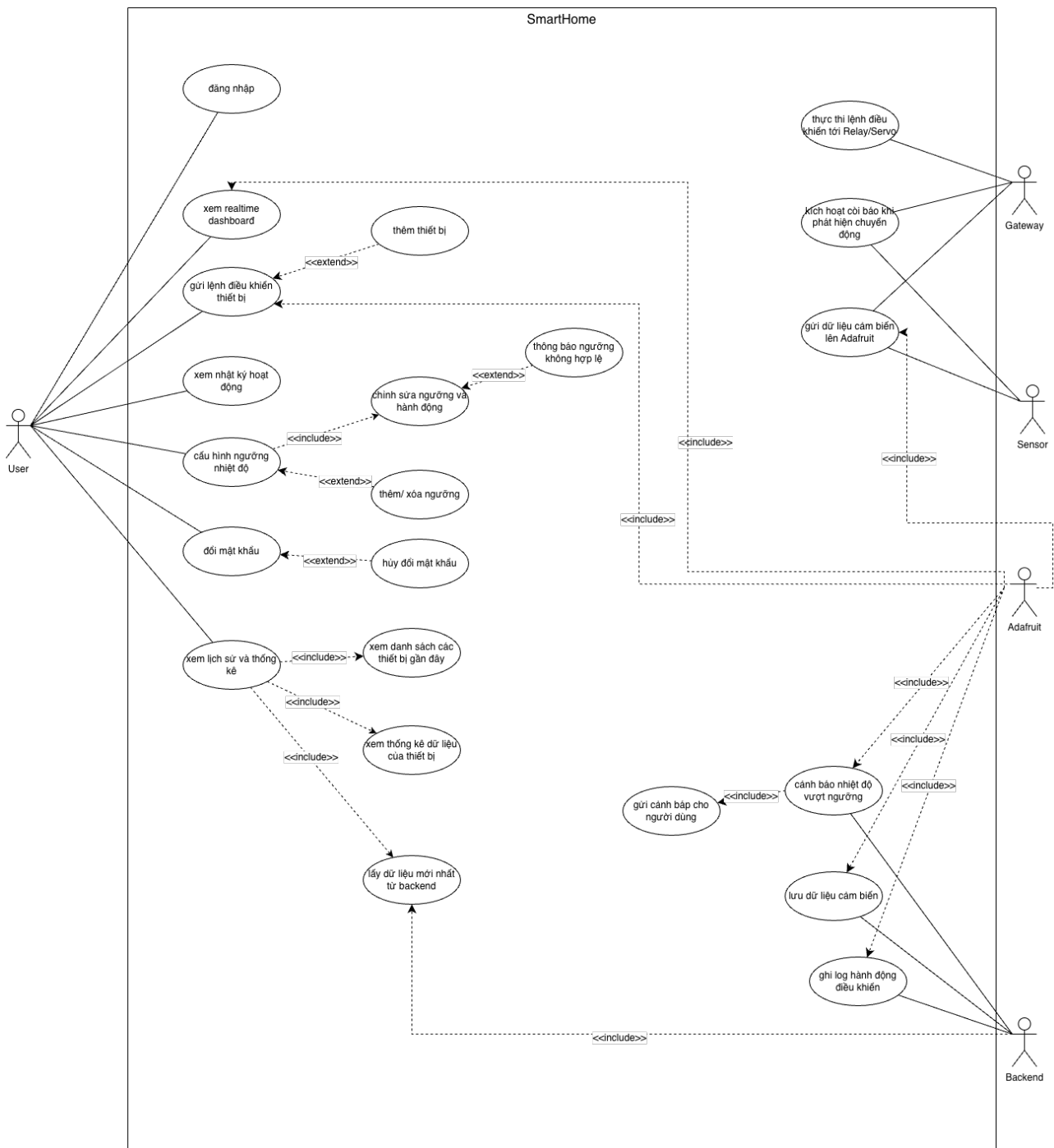
Backend đóng vai trò như một subscriber kết nối với server Adafruit thông qua giao thức MQTT. Hệ thống sẽ tự động nhận dữ liệu từ các cảm biến, xử lý và lưu trữ vào cơ sở dữ liệu để phục vụ cho việc hiển thị, phân tích và truy xuất sau này.

1.6.2 Yêu cầu phi chức năng

Các yêu cầu phi chức năng mô tả các tiêu chí về chất lượng mà hệ thống cần đáp ứng.

- **Hiệu năng (Performance):** Hệ thống phải đảm bảo khả năng xử lý và cập nhật dữ liệu theo thời gian thực với độ trễ thấp.
- **Độ tin cậy (Reliability):** Hệ thống phải hoạt động ổn định, đảm bảo không mất dữ liệu trong quá trình thu thập và lưu trữ.
- **Bảo mật (Security):** Thông tin người dùng, đặc biệt là mật khẩu, phải được mã hóa. Hệ thống cần có cơ chế xác thực và phân quyền phù hợp.
- **Khả năng mở rộng (Scalability):** Hệ thống có khả năng mở rộng để hỗ trợ thêm nhiều thiết bị và người dùng trong tương lai.
- **Khả năng sử dụng (Usability):** Giao diện người dùng phải thân thiện, dễ sử dụng và hỗ trợ thao tác nhanh chóng.
- **Khả năng bảo trì (Maintainability):** Hệ thống được thiết kế theo hướng dễ bảo trì, dễ nâng cấp và mở rộng chức năng khi cần thiết.

1.7 Use-case Diagram



Hình 1: Usecase Diagram cho Yolo:Home

Use case diagram của hệ thống Yolo:Home mô tả các tương tác chính giữa người dùng và các thành phần trong hệ thống IoT, bao gồm User, Sensor, IoT Gateway, Adafruit IO và Backend. Sơ đồ thể hiện đầy đủ các chức năng chính của hệ thống, từ thu thập dữ liệu, xử lý, đến hiển

thị và điều khiển thiết bị.

Actor User

Người dùng là tác nhân trung tâm, tương tác trực tiếp với hệ thống thông qua giao diện web. Người dùng có thể thực hiện các chức năng như đăng nhập, xem dashboard thời gian thực, gửi lệnh điều khiển thiết bị, xem lịch sử và thống kê dữ liệu, cấu hình ngưỡng nhiệt độ và thay đổi mật khẩu.

Khi đăng nhập thành công, người dùng có thể truy cập dashboard để quan sát dữ liệu cảm biến theo thời gian thực. Ngoài ra, người dùng có thể gửi lệnh điều khiển thiết bị như bật/tắt quạt hoặc đèn. Trong quá trình cấu hình ngưỡng nhiệt độ, hệ thống cho phép chỉnh sửa ngưỡng và hành động tương ứng; nếu giá trị không hợp lệ, use case “thông báo ngưỡng không hợp lệ” sẽ được kích hoạt như một trường hợp mở rộng (extend).

Use case “xem lịch sử và thống kê” cho phép người dùng xem dữ liệu quá khứ của hệ thống, bao gồm danh sách thiết bị gần đây và thống kê dữ liệu của từng thiết bị. Ngoài ra, người dùng cũng có thể yêu cầu hệ thống lấy dữ liệu mới nhất từ backend để cập nhật thông tin.

Actor Sensor

Sensor chịu trách nhiệm thu thập dữ liệu môi trường. Trong sơ đồ, sensor thực hiện hai chức năng chính là gửi dữ liệu cảm biến và phát hiện chuyển động. Dữ liệu thu thập được sẽ được truyền lên hệ thống thông qua Gateway và Adafruit IO để phục vụ việc hiển thị và xử lý.

Actor IoT Gateway

IoT Gateway đóng vai trò trung gian giữa phần cứng và hệ thống mạng. Gateway nhận dữ liệu từ sensor và gửi dữ liệu cảm biến lên Adafruit IO. Đồng thời, khi nhận lệnh điều khiển từ hệ thống, Gateway sẽ thực thi lệnh điều khiển tới các thiết bị chấp hành như relay hoặc servo.

Ngoài ra, Gateway còn có chức năng kích hoạt còi báo động khi phát hiện chuyển động, đảm bảo khả năng phản ứng nhanh tại thiết bị mà không cần phụ thuộc hoàn toàn vào backend.

Actor Adafruit IO (MQTT Broker)

Adafruit IO đóng vai trò là hệ thống trung gian truyền thông, sử dụng giao thức MQTT để kết nối các thành phần. Khi nhận dữ liệu cảm biến, Adafruit IO sẽ chuyển dữ liệu đến dashboard để hiển thị realtime, đồng thời chuyển đến backend để lưu trữ và xử lý.

Khi người dùng gửi lệnh điều khiển, Adafruit IO tiếp nhận lệnh và chuyển đến Gateway để thực thi. Nhờ cơ chế publish-subscribe, hệ thống đảm bảo truyền dữ liệu theo thời gian thực với độ trễ thấp.

Actor Backend

Backend chịu trách nhiệm xử lý logic và lưu trữ dữ liệu. Khi nhận dữ liệu từ Adafruit IO, backend sẽ lưu dữ liệu cảm biến vào cơ sở dữ liệu và ghi log các hành động điều khiển.

Một chức năng quan trọng của backend là phát hiện nhiệt độ vượt ngưỡng. Khi điều kiện này xảy ra, backend kích hoạt use case “cảnh báo nhiệt độ vượt ngưỡng” và gửi cảnh báo đến người dùng.

Ngoài ra, backend còn cung cấp dữ liệu lịch sử và thống kê cho người dùng thông qua các API, đồng thời hỗ trợ use case “lấy dữ liệu mới nhất từ backend”.

1.8 Use-case Scenarios

1.8.1 Module 1: Trực quan hóa dữ liệu (Data Visualization)

Use-case 01: Send sensor data to Adafruit

Use-case name	UC-01: Send sensor data to Adafruit
Actors	Yolo:Bit + Sensors (DHT20/PIR), IoT Gateway (Python), Adafruit IO (MQTT Broker)
Goal	Đưa dữ liệu cảm biến (nhiệt độ/độ ẩm/chuyển động) lên Adafruit để hệ thống đọc realtime và lưu trữ.
Preconditions	• Thiết bị và cảm biến hoạt động. • Gateway kết nối Internet. • Feed/credentials Adafruit hợp lệ.
Trigger	Cảm biến phát sinh dữ liệu theo chu kỳ hoặc PIR phát hiện chuyển động.
Main flow	1. Yolo:Bit đọc dữ liệu từ DHT20/PIR. 2. Yolo:Bit gửi dữ liệu sang Gateway qua Serial/UART. 3. Gateway chuẩn hóa/đóng gói dữ liệu (ví dụ JSON/field-value). 4. Gateway publish dữ liệu lên Adafruit IO qua MQTT vào đúng feed/topic. 5. Adafruit IO nhận và lưu bản tin để các subscriber sử dụng.
Alternative flow	A4.1: Mất mạng tạm thời 1. Ở bước 4, Gateway không publish được dữ liệu lên Adafruit IO. 2. Gateway lưu đệm dữ liệu cục bộ. 3. Gateway retry publish theo chu kỳ/backoff khi mạng hồi phục.

	4. Khi publish thành công, tiếp tục về bước 5 của main flow.
Exception flow	E1: Lỗi đọc cảm biến 1. Gateway log lỗi và bỏ qua chu kỳ hiện tại hoặc publish trạng thái <code>sensor_error</code> . E2: Sai key/username Adafruit 1. Gateway publish thất bại và cảnh báo lỗi cấu hình.
Post-conditions	Dữ liệu cảm biến xuất hiện trên Adafruit IO để Frontend/Backend subscribe và xử lý.
Extension points	None

Use-case 02: Subscribe realtime data on Dashboard

Use-case name	UC-02: Subscribe realtime data on Dashboard
Actors	User, Frontend Web/React, Adafruit IO (MQTT Broker)
Goal	Hiển thị dữ liệu môi trường realtime trên Dashboard.
Preconditions	- User đang ở màn hình Dashboard. - Frontend có thể kết nối Adafruit và có quyền subscribe feed/topic.
Trigger	App vừa load trang Dashboard.
Main flow	1. Frontend khởi tạo kết nối MQTT/WebSocket đến Adafruit. 2. Frontend subscribe các topic dữ liệu (temp/humidity/motion). 3. Adafruit đẩy bản tin mới khi có publish từ UC-01. 4. Frontend parse payload. 5. Frontend cập nhật UI realtime (số liệu/trạng thái).
Alternative flow	A5.1: User rời Dashboard

	1. Sau bước 5, Frontend ngừng render realtime và có thể disconnect khỏi Adafruit. 2. Khi user quay lại Dashboard, Frontend quay lại bước 1 để kết nối lại. 3. Frontend tiếp tục bước 2 để subscribe lại các topic.
Exception flow	E1: Không kết nối được Adafruit 1. UI báo “mất kết nối realtime” và cho phép retry. E2: Payload sai định dạng 1. Frontend bỏ qua bản tin và ghi log lỗi.
Post-conditions	Dashboard hiển thị dữ liệu realtime thành công.
Extension points	None

Use-case 03: Fetch latest stats from Backend API

Use-case name	UC-03: Fetch latest stats from Backend API
Actors	User, Frontend Web/React, Backend API, Database (PostgreSQL)
Goal	Tải dữ liệu snapshot/thống kê gần nhất để hiển thị ngay khi mở app (không phụ thuộc realtime).
Preconditions	• Backend đang chạy và DB có dữ liệu. • Frontend có thể gọi API (base URL hợp lệ, auth).
Trigger	App/Web load xong trang Dashboard.
Main flow	1. Frontend gửi request API “get latest stats” đến Backend. 2. Backend xác thực request. 3. Backend query DB lấy bản ghi mới nhất và/hoặc thống kê nhanh. 4. Backend trả response JSON. 5. Frontend render dữ liệu lên Dashboard.
Alternative flow	A3.1: DB chưa có dữ liệu

	- Ở bước 3, Backend không tìm thấy bản ghi phù hợp. - Backend trả response “empty state” ở bước 4. - Frontend hiển thị trạng thái “chưa có dữ liệu/đang chờ dữ liệu” ở bước 5.
Exception flow	E1: API timeout/lỗi server Frontend hiển thị lỗi tải dữ liệu và nút Retry.
Post-conditions	Dashboard có dữ liệu khởi tạo (initial data) sau khi mở app.
Extension points	None

1.8.2 Module 2: Giám sát và tự động xử lý dữ liệu (Monitoring & Automation)

Use-case 04: Temperature threshold alert (> 35°C)

Use-case name	UC-04: Temperature threshold alert (> 35°C)
Actors	Backend (MQTT subscriber), Adafruit IO, Frontend
Goal	Tự động phát cảnh báo khi nhiệt độ vượt ngưỡng 35°C.
Preconditions	- Backend đang subscribe feed nhiệt độ trên Adafruit. - Frontend có cơ chế nhận/hiển thị alert.
Trigger	Backend nhận bản tin nhiệt độ mới từ Adafruit.
Main flow	- Backend nhận message nhiệt độ từ Adafruit. - Backend parse giá trị nhiệt độ. - Backend so sánh với ngưỡng 35°C. - Nếu vượt ngưỡng, Backend tạo alert event. - Backend gửi alert xuống Frontend (kèm giá trị/thời gian/mức độ). - Frontend hiển thị cảnh báo cho user.
Alternative flow	A4.1: Tránh spam cảnh báo Ở bước 4, trước khi tạo alert event, Backend kiểm tra cooldown/hysteresis.

	2. Nếu đang trong thời gian cooldown hoặc chưa thỏa điều kiện cảnh báo lại, Backend không tạo alert mới. 3. Use case kết thúc mà không thực hiện bước 5 và bước 6. 4. Nếu đủ điều kiện, tiếp tục bước 5.
Exception flow	E1: Backend mất kết nối MQTT 1. Backend ghi log lỗi và tự reconnect. E2: Frontend offline 1. Backend lưu hàng đợi hoặc ghi log để xử lý sau.
Post-conditions	User nhận được cảnh báo nhiệt độ bất thường.
Extension points	Thay đổi ngưỡng cảnh báo theo cấu hình.

Use-case 05: Motion alarm (PIR) triggers siren

Use-case name	UC-05: Motion alarm (PIR) triggers siren
Actors	PIR Sensor, Yolo:Bit, IoT Gateway, Còi/Buzzer (thiết bị báo động)
Goal	Tự động hóa bảo mật tại chỗ: phát còi khi có chuyển động trong chế độ cần bảo mật.
Preconditions	- PIR hoạt động ổn định. - Chế độ “ban đêm” hoặc “away mode” đang bật. - Gateway điều khiển được còi qua Yolo:Bit.
Trigger	PIR phát hiện chuyển động.
Main flow	1. PIR phát hiện chuyển động và gửi tín hiệu về Yolo:Bit. 2. Yolo:Bit gửi event sang Gateway. 3. Gateway kiểm tra điều kiện (ban đêm/away mode). 4. Nếu thỏa điều kiện, Gateway gửi lệnh kích hoạt còi xuống Yolo:Bit. 5. Yolo:Bit bật còi theo thời gian cấu hình.

	6. Gateway ghi log sự kiện.
Alternative flow	A3.1: Đang ở “home mode” 1. Ở bước 3, Gateway xác định hệ thống đang ở home mode. 2. Gateway không thực hiện bước 4 và bước 5. 3. Gateway chỉ ghi nhận chuyển động và tiếp tục bước 6 để log sự kiện.
Exception flow	E1: PIR nhiễu/false positive 1. Gateway debounce hoặc lọc thời gian tối thiểu giữa 2 lần kích hoạt. E2: Lỗi điều khiển còi 1. Gateway ghi log lỗi phản cứng và dừng kích hoạt.
Post-conditions	Còi báo động được kích hoạt đúng điều kiện, sự kiện được ghi nhận.
Extension points	None

1.8.3 Module 3: Điều khiển thiết bị (Device Control)

Use-case 06: User publishes control command (Turn ON fan)

Use-case name	UC-06: User publishes control command (Turn ON fan)
Actors	User, Frontend Web/React, Adafruit IO
Goal	Gửi lệnh điều khiển thiết bị (ví dụ bật quạt) lên kênh điều khiển thông qua Adafruit.
Preconditions	• User có quyền điều khiển thiết bị. • Frontend kết nối được Adafruit và biết topic điều khiển. • Format lệnh điều khiển được hệ thống quy ước (JSON).

Trigger	User nhấn nút “Bật quạt”.
Main flow	1. User nhấn “Bật quạt”. 2. Frontend tạo payload lệnh điều khiển (JSON: device=fan, action=on, timestamp...). 3. Frontend publish payload lên topic điều khiển trên Adafruit. 4. Adafruit nhận message và lưu để Gateway subscribe.
Alternative flow	A1.1: User nhấn liên tục 1. Tại bước 1, Frontend phát hiện nhiều thao tác nhấn liên tiếp. 2. Frontend áp dụng cooldown hoặc disable nút tạm thời. 3. Chỉ giữ lại thao tác cuối cùng để tiếp tục bước 2.
Exception flow	E1: Publish thất bại 1. Frontend báo lỗi và cho phép Retry. E2: Payload không hợp lệ 1. Frontend validate và chặn gửi, hiển thị lỗi input.
Post-conditions	Lệnh điều khiển xuất hiện trên Adafruit topic điều khiển.
Extension points	None

Use-case 07: Gateway executes command to Relay/Servo

Use-case name	UC-07: Gateway executes command to Relay/Servo
Actors	Adafruit IO, IoT Gateway, Yolo:Bit, Relay/Servo
Goal	Thực thi lệnh điều khiển lên thiết bị vật lý (bật quạt qua Relay / mở khóa qua Servo).
Preconditions	• Gateway đang subscribe topic điều khiển. • Yolo:Bit kết nối ổn định với Gateway.

	• Relay/Servo hoạt động bình thường.
Trigger	Có bản tin điều khiển mới trên Adafruit.
Main flow	1. Gateway nhận message điều khiển từ Adafruit. 2. Gateway parse JSON và xác định thiết bị đích + hành động. 3. Gateway chuyển đổi lệnh sang giao thức điều khiển nội bộ cho Yolo:Bit. 4. Gateway gửi lệnh xuống Yolo:Bit qua Serial. 5. Yolo:Bit kích Relay hoặc điều khiển Servo theo yêu cầu. 6. Gateway publish trạng thái mới (device_state) lên Adafruit để Frontend cập nhật.
Alternative flow	A2.1: Thiết bị đã ở đúng trạng thái 1. Ở bước 2, Gateway xác định thiết bị đã ở trạng thái được yêu cầu. 2. Gateway bỏ qua bước 3, bước 4, bước 5. 3. Gateway thực hiện bước 6 để publish trạng thái hiện tại lên Adafruit.
Exception flow	E1: MQTT disconnect 1. Gateway tự reconnect và ghi log lỗi. E2: Yolo:Bit không phản hồi 1. Gateway retry N lần, sau đó báo lỗi thiết bị. E3: Relay/Servo lỗi hoặc bị kẹt 1. Gateway dừng thao tác và ghi log lỗi để an toàn.
Post-conditions	Thiết bị vật lý được điều khiển đúng theo lệnh, trạng thái hệ thống được cập nhật.
Extension points	None

1.8.4 Module 4: Ghi nhận hoạt động

Use-case 08: Backend stores sensor data periodically

Use-case name	UC-08: Backend stores sensor data periodically
Actors	Backend (MQTT subscriber), Adafruit IO, Database (PostgreSQL/MongoDB)
Goal	Tự động lưu trữ dữ liệu cảm biến từ Adafruit IO vào Database để phục vụ thống kê và lịch sử.
Preconditions	• Backend đang subscribe các feed cảm biến (nhiệt độ, độ ẩm, PIR) trên Adafruit. • Database đã được cấu hình và kết nối. • Có cấu hình chu kỳ lưu (VD: 1 phút/lần) hoặc lưu mỗi khi có dữ liệu mới.
Trigger	Backend nhận được bản tin mới từ Adafruit (hoặc hết chu kỳ thời gian nếu lưu theo batch).
Main flow	1. Backend nhận message từ Adafruit qua MQTT. 2. Backend parse payload để lấy các giá trị: timestamp (nếu có), nhiệt độ, độ ẩm, trạng thái PIR. 3. Backend kết nối đến Database. 4. Backend thực hiện INSERT dữ liệu vào bảng sensor_data (các trường: id, timestamp, temperature, humidity, motion, device_id). 5. Backend commit transaction và đóng kết nối. 6. Backend ghi log thành công (hoặc log debug).
Alternative flow	A1: Lưu theo batch (chu kỳ) 1. Backend không lưu ngay mỗi khi có dữ liệu mà tích lũy trong bộ nhớ đệm.

	2. Sau mỗi X phút, Backend gom tất cả dữ liệu trong đệm và INSERT hàng loạt (batch insert) để tối ưu hiệu năng. 3. Xóa đệm sau khi lưu thành công.
Exception flow	E1: Mất kết nối Database 1. Backend không thể INSERT dữ liệu. 2. Backend lưu dữ liệu vào hàng đợi (queue) hoặc file tạm. 3. Backend retry kết nối Database theo chu kỳ. 4. Khi Database hoạt động lại, Backend đẩy dữ liệu từ hàng đợi vào DB. E2: Payload không hợp lệ hoặc thiếu trường 1. Backend ghi log lỗi parsing. 2. Bỏ qua bản tin đó và tiếp tục xử lý bản tin tiếp theo.
Post-conditions	Dữ liệu cảm biến được lưu trữ an toàn trong Database, sẵn sàng cho các truy vấn thống kê và lịch sử.
Extension points	None

Use-case 09: Backend logs all control actions

Use-case name	UC-09: Backend logs all control actions
Actors	Backend (MQTT subscriber), Adafruit IO, Database
Goal	Ghi lại lịch sử mọi hành động điều khiển thiết bị (từ người dùng hoặc tự động) để phục vụ audit và xem lại.
Preconditions	- Backend subscribe các feed điều khiển (control commands) và feed trạng thái thiết bị. - Database có bảng <code>action_log</code>.
Trigger	Có một lệnh điều khiển được publish lên Adafruit (từ UC-06) hoặc có hành động tự động từ Gateway (UC-05).

Main flow	1. Backend nhận message từ feed điều khiển (hoặc từ feed trạng thái thiết bị sau khi thực thi). 2. Backend parse payload để xác định: thiết bị (device), hành động (action), thời gian (timestamp), nguồn (source: "user"/ "auto"). 3. Backend INSERT bản ghi vào bảng <code>action_log</code> (các trường: id, timestamp, device, action, source, status). 4. Backend commit và đóng kết nối. 5. Backend ghi log debug (nếu cần).
Alternative flow	A1: Log từ nhiều nguồn 1. Nếu hành động xuất phát từ tự động hóa cục bộ (UC-05), Gateway publish trạng thái lên Adafruit. 2. Backend subscribe trạng thái đó và ghi log tương tự. A2: Ghi log thất bại 1. Nếu thiết bị không phản hồi hoặc lệnh thất bại, Backend vẫn ghi log với status = "failed".
Exception flow	E1: Không xác định được thiết bị/hành động 1. Backend ghi log lỗi parsing. 2. Bỏ qua bản tin đó. E2: Lỗi Database 1. Backend ghi log lỗi và lưu tạm vào bộ nhớ. 2. Retry khi Database hoạt động lại.
Post-conditions	Mọi hành động điều khiển đều có dấu vết trong Database, phục vụ cho việc tra cứu lịch sử và phân tích.
Extension points	None

Use-case 10: User views history and statistics

Use-case name	UC-10: User views history and statistics
Actors	User, Frontend (Mobile App), Backend API, Database
Goal	Cho phép người dùng xem lại lịch sử dữ liệu cảm biến và các thao tác điều khiển dưới dạng biểu đồ và danh sách.
Preconditions	• User đã đăng nhập (có JWT token). • Database có dữ liệu từ UC-08 và UC-09. • Backend API hoạt động.
Trigger	User vào mục "Lịch sử"(History) trên ứng dụng.
Main flow	1. User chọn mục "Lịch sử"trên giao diện. 2. Frontend hiển thị màn hình với các tùy chọn: xem theo ngày, theo thiết bị, theo loại dữ liệu. 3. User chọn khoảng thời gian (VD: hôm nay, 7 ngày qua, tùy chọn). 4. Frontend gửi request API GET /api/history kèm tham số thời gian và token. 5. Backend xác thực token và kiểm tra quyền. 6. Backend query Database để lấy: - Dữ liệu cảm biến trong khoảng thời gian (temperature, humidity, motion). - Danh sách các action log (bật/tắt thiết bị, cảnh báo). 7. Backend trả về JSON chứa dữ liệu đã được tổng hợp. 8. Frontend render: - Biểu đồ đường (line chart) cho nhiệt độ/độ ẩm. - Danh sách các sự kiện (dạng bảng hoặc card). 9. Frontend cho phép user lọc theo loại sự kiện (chỉ xem cảnh báo, chỉ xem điều khiển).
Alternative flow	A1: Không có dữ liệu trong khoảng thời gian 1. Backend trả về mảng rỗng.

	2. Frontend hiển thị thông báo "Không có dữ liệu trong khoảng thời gian này". A2: Xuất dữ liệu (nếu có) 1. Frontend có thêm nút "Xuất báo cáo". 2. Gọi API xuất file (CSV/PDF) và tải về.
Exception flow	E1: Token hết hạn/không hợp lệ 1. Backend trả về 401 Unauthorized. 2. Frontend chuyển hướng về màn hình đăng nhập. E2: API timeout/lỗi server 1. Frontend hiển thị thông báo lỗi và nút "Thử lại".
Post-conditions	User xem được lịch sử và thống kê mong muốn.
Extension points	None

1.8.5 Module 5: Ứng dụng Web

Use-case 11: User logs in

Use-case name	UC-11: User logs in
Actors	User, Frontend, Backend API, Database (users table)
Goal	Xác thực người dùng và cấp quyền truy cập vào hệ thống.
Preconditions	• User đã có tài khoản (đã đăng ký trước đó). • Backend API hoạt động.
Trigger	User mở ứng dụng và chưa đăng nhập (hoặc token hết hạn).
Main flow	1. Frontend hiển thị màn hình đăng nhập (username/email và password). 2. User nhập thông tin và nhấn "Đăng nhập". 3. Frontend gửi request POST /api/auth/login với credentials.

	4. Backend kiểm tra thông tin trong Database (hash password). 5. Nếu hợp lệ, Backend tạo JWT token (có thời hạn) và refresh token (tùy chọn). 6. Backend trả về token kèm thông tin user cơ bản. 7. Frontend lưu token vào secure storage (SharedPreferences/Keychain). 8. Frontend chuyển đến màn hình chính (Dashboard).
	A1: Duy trì đăng nhập 1. Khi mở app lần sau, Frontend kiểm tra token còn hạn không. 2. Nếu còn, tự động đăng nhập và vào thẳng Dashboard.
Exception flow	E1: Sai tài khoản/mật khẩu 1. Backend trả về 401. 2. Frontend hiển thị thông báo lỗi. E2: Tài khoản bị khóa 1. Backend trả về lỗi kèm message. 2. Frontend hướng dẫn user liên hệ hỗ trợ.
Post-conditions	User được xác thực và có quyền truy cập vào các chức năng khác.
Extension points	None

Use-case 12: User configures temperature alert threshold

Use-case name	UC-12: User configures temperature alert threshold
Actors	User, Frontend, Backend API, Database (user_config table)
Goal	Cho phép người dùng thay đổi ngưỡng cảnh báo nhiệt độ theo nhu cầu cá nhân.
Preconditions	User đã đăng nhập.

	• Backend API hoạt động.
Trigger	User vào mục "Cài đặt"(Settings) và chọn "Ngưỡng cảnh báo nhiệt độ".
Main flow	1. Frontend hiển thị màn hình cấu hình với giá trị ngưỡng hiện tại (lấy từ API). 2. User nhập giá trị mới (VD: 33°C) và nhấn "Lưu". 3. Frontend gửi request PUT /api/user/config với payload { "temp_threshold": 33 } kèm token. 4. Backend xác thực token và kiểm tra giá trị hợp lệ (ví dụ: trong khoảng 20-50°C). 5. Backend cập nhật giá trị vào Database (bảng user_config). 6. Backend trả về success response. 7. Frontend hiển thị thông báo "Lưu thành công". 8. (<i>Quan trọng</i>) Backend cập nhật ngưỡng mới cho các tiến trình giám sát (UC-04) để sử dụng ngay lập tức.
Alternative flow	A1: Hủy thay đổi 1. User nhấn "Hủy" hoặc back. 2. Frontend không gửi request, quay lại màn hình trước.
Exception flow	E1: Giá trị không hợp lệ (quá cao/quá thấp) 1. Backend trả về 400 với message lỗi. 2. Frontend hiển thị lỗi và yêu cầu nhập lại. E2: Lỗi Database 1. Backend trả về 500. 2. Frontend thông báo lỗi hệ thống, yêu cầu thử lại sau.
Post-conditions	Ngưỡng cảnh báo mới được áp dụng cho toàn bộ hệ thống.
Extension points	None

Use-case 13: User changes account password

Use-case name	UC-13: User changes account password
Actors	User, Frontend, Backend API, Database
Goal	Cho phép người dùng thay đổi mật khẩu tài khoản để bảo mật.
Preconditions	• User đã đăng nhập. • Backend API hoạt động.
Trigger	User vào mục "Cài đặt" và chọn "Đổi mật khẩu".
Main flow	1. Frontend hiển thị form đổi mật khẩu gồm 3 trường: mật khẩu cũ, mật khẩu mới, xác nhận mật khẩu mới. 2. User nhập đầy đủ và nhấn "Xác nhận". 3. Frontend kiểm tra cơ bản: mật khẩu mới và xác nhận phải khớp, độ dài tối thiểu (VD: 6 ký tự). 4. Frontend gửi request POST /api/auth/change-password kèm token và payload { "old_password": "...", "new_password": "..."}. 5. Backend xác thực token và lấy user ID. 6. Backend kiểm tra mật khẩu cũ có khớp với hash trong Database không. 7. Nếu khớp, Backend hash mật khẩu mới và cập nhật vào Database. 8. Backend trả về success response. 9. Frontend hiển thị thông báo "Đổi mật khẩu thành công". 10. (Tùy chọn) Backend vô hiệu hóa các token cũ (nếu muốn) hoặc giữ nguyên.
Alternative flow	A1: Hủy thay đổi 1. User nhấn "Hủy" hoặc back.

	2. Frontend không gửi request, quay lại màn hình trước.
Exception flow	E1: Mật khẩu cũ không đúng - Backend trả về 401 với message "Mật khẩu cũ không chính xác". - Frontend hiển thị lỗi và yêu cầu nhập lại. E2: Mật khẩu mới không đủ mạnh - Backend kiểm tra độ mạnh và trả về 400 nếu không đạt. - Frontend hiển thị yêu cầu cụ thể (ít nhất 1 số, 1 chữ hoa, v.v.). E3: Lỗi Database - Backend trả về 500. - Frontend thông báo lỗi hệ thống.
Post-conditions	Mật khẩu mới được cập nhật, user có thể đăng nhập bằng mật khẩu mới ở lần tiếp theo.
Extension points	None

1.9 Phân tích nghiệp vụ hệ thống

Hệ thống nhà thông minh được xây dựng nhằm hỗ trợ người dùng trong việc quản lý và điều khiển các thiết bị trong ngôi nhà một cách tiện lợi, linh hoạt và hiệu quả. Thông qua hệ thống, người dùng có thể theo dõi trạng thái thiết bị, thu thập dữ liệu từ các cảm biến môi trường và thiết lập các cơ chế tự động hóa phù hợp với nhu cầu sử dụng thực tế. Việc phân tích nghiệp vụ đóng vai trò quan trọng trong việc xác định rõ các chức năng, luồng hoạt động và các quy tắc vận hành của hệ thống, từ đó làm cơ sở cho quá trình thiết kế và triển khai.

Quản lý người dùng

Trong hệ thống này, người dùng là tác nhân trực tiếp tương tác với hệ thống. Người dùng cần thực hiện đăng ký tài khoản bằng các thông tin như email, mật khẩu (password) và họ tên (full_name), sau đó đăng nhập để truy cập vào hệ thống. Sau khi đăng nhập thành công, người dùng có thể quản lý toàn bộ các thành phần trong ngôi nhà của mình thông qua một giao diện tập trung. Ngoài ra, người dùng có thể tạo các phòng mới để tổ chức và quản lý các thiết bị, cũng như thực hiện các thao tác như thêm, xóa và chỉnh sửa thiết bị trong từng phòng.

Quản lý phòng

Đối với chức năng quản lý phòng, người dùng có thể tạo mới, chỉnh sửa hoặc xóa các phòng trong hệ thống. Khi tạo phòng, người dùng chỉ cần cung cấp tên phòng (room_name), hệ thống sẽ tự động tạo một mã định danh (room_id) duy nhất để quản lý. Các phòng đóng vai trò là nơi chứa các thiết bị, giúp tổ chức hệ thống theo cấu trúc logic và dễ dàng quản lý hơn. Một người dùng có thể sở hữu nhiều phòng, và mỗi phòng có thể chứa nhiều thiết bị khác nhau.

Quản lý thiết bị

Chức năng quản lý thiết bị là một trong những nghiệp vụ cốt lõi của hệ thống. Người dùng có thể thêm thiết bị vào từng phòng cụ thể bằng cách cung cấp các thông tin như tên thiết bị (device_name), loại thiết bị (type) và thời gian cài đặt (Install_date). Bên cạnh đó, hệ thống sẽ tự động cấp mã định danh cho thiết bị (device_id) và kiểm tra trạng thái kết nối (state) nhằm đảm bảo việc quản lý chính xác.

Trong hệ thống, thiết bị được chia thành hai loại chính là thiết bị đầu ra (Output_device) và thiết bị cảm biến (Sensor). Thiết bị đầu ra bao gồm các thiết bị có thể điều khiển như đèn, quạt hoặc máy lạnh, với các trạng thái như bật/tắt (ON/OFF) hoặc chế độ tự động (AUTO). Trong khi đó, thiết bị cảm biến có nhiệm vụ thu thập dữ liệu từ môi trường như nhiệt độ, độ ẩm và thường được cấu hình với các ngưỡng giá trị tối thiểu (threshold_min) và tối đa (threshold_max) để phục vụ cho việc tự động hóa.

Hệ thống cho phép người dùng điều khiển thiết bị một cách trực tiếp thông qua các thao tác như bật, tắt hoặc chuyển đổi chế độ hoạt động. Khi người dùng thực hiện một hành động điều khiển, hệ thống sẽ tiếp nhận yêu cầu, gửi lệnh tới thiết bị tương ứng và cập nhật lại trạng thái của thiết bị trong cơ sở dữ liệu. Đồng thời, mọi hành động này đều được ghi nhận lại nhằm phục vụ cho việc theo dõi và kiểm tra sau này.

Ghi nhận lịch sử hoạt động

Bên cạnh việc điều khiển thủ công, hệ thống còn hỗ trợ chức năng ghi nhận lịch sử hoạt động thông qua bảng nhật ký (Action_logs). Mỗi khi có một hành động xảy ra, có thể là do người dùng thực hiện hay do hệ thống tự động kích hoạt, một bản ghi log sẽ được tạo ra với các thông tin như thời gian thực hiện (timestamp), loại hành động (log_type), tên thiết bị (device_name), nội dung hành động (action) và các chi tiết liên quan (detail). Việc lưu trữ các log này giúp người dùng có thể theo dõi lại quá trình hoạt động của hệ thống, đồng thời hỗ trợ cho việc phân tích và xử lý sự cố khi cần thiết.

Thu thập dữ liệu cảm biến

Đối với các thiết bị cảm biến, hệ thống thực hiện việc thu thập dữ liệu một cách liên tục theo thời gian thực. Mỗi cảm biến sẽ định kỳ gửi các giá trị đo được về hệ thống, bao gồm giá trị đo (value) và thời điểm đo (time). Các dữ liệu này được lưu trữ trong bảng Sensor_data và có thể được sử dụng để hiển thị dưới dạng biểu đồ, phục vụ cho việc giám sát môi trường sống. Ngoài ra, dữ liệu cảm biến còn đóng vai trò quan trọng trong việc kích hoạt các chức năng tự động hóa của hệ thống.

Tự động hóa hệ thống

Chức năng tự động hóa cho phép hệ thống tự động điều khiển các thiết bị đầu ra dựa trên dữ liệu thu thập từ cảm biến. Cụ thể, mỗi cảm biến sẽ được thiết lập một ngưỡng giá trị tối thiểu và tối đa. Khi giá trị đo được vượt ra ngoài khoảng ngưỡng này, hệ thống sẽ tự động kích hoạt các thiết bị liên quan, ví dụ như bật quạt khi nhiệt độ quá cao hoặc tắt quạt khi nhiệt độ giảm xuống dưới ngưỡng cho phép. Sau khi thực hiện hành động, hệ thống cũng sẽ ghi nhận lại vào bảng log tương tự như các thao tác thủ công.

Luồng hoạt động tổng thể

Tổng thể, luồng hoạt động của hệ thống bắt đầu từ việc người dùng đăng nhập, sau đó tạo và quản lý các phòng, thêm thiết bị vào từng phòng và theo dõi hoạt động của các thiết bị này. Các thiết bị cảm biến liên tục gửi dữ liệu về hệ thống, trong khi các thiết bị đầu ra nhận lệnh từ người dùng hoặc từ cơ chế tự động hóa. Tất cả các hành động đều được ghi nhận lại nhằm đảm bảo tính minh bạch và khả năng truy vết của hệ thống.

2 Thiết kế hệ thống

2.1 Lựa chọn mẫu thiết kế

Trong quá trình thiết kế hệ thống Yolo:Home, việc lựa chọn các mẫu thiết kế phù hợp có vai trò quan trọng trong việc hình thành một kiến trúc rõ ràng, dễ mở rộng và dễ bảo trì. Do hệ thống vừa có đặc trưng của một ứng dụng web quản lý dữ liệu, vừa có đặc trưng của một hệ thống IoT xử lý dữ liệu thời gian thực, nên các mẫu thiết kế được lựa chọn không chỉ nhằm tổ chức mã nguồn hợp lý mà còn để giải quyết những vấn đề thực tế phát sinh trong quá trình vận hành. Ba mẫu thiết kế chính được áp dụng bao gồm **Repository Pattern**, **Dependency Injection** và **Observer Pattern**.

2.1.1 Repository Pattern

Khái quát về mẫu thiết kế

Repository Pattern là một mẫu thiết kế được sử dụng để trừu tượng hóa tầng truy cập dữ liệu. Thay vì để các thành phần xử lý nghiệp vụ thao tác trực tiếp với cơ sở dữ liệu hoặc công nghệ lưu trữ cụ thể, hệ thống sử dụng một lớp trung gian gọi là repository để thực hiện các thao tác đọc, ghi, cập nhật và xóa dữ liệu. Nhờ đó, phần logic nghiệp vụ chỉ làm việc với dữ liệu ở mức khái niệm, không cần quan tâm chi tiết việc dữ liệu được lưu trữ hay truy xuất như thế nào.

Về bản chất, Repository Pattern giúp tạo ra một ranh giới rõ ràng giữa tầng nghiệp vụ và tầng dữ liệu. Mẫu này đặc biệt hữu ích trong các hệ thống có nhiều thực thể và nhiều thao tác truy vấn khác nhau, nơi việc để logic nghiệp vụ truy cập trực tiếp vào cơ sở dữ liệu sẽ làm cho kiến trúc trở nên rối và khó kiểm soát.

Cấu trúc của Repository Pattern

Cấu trúc của Repository Pattern thường bao gồm hai phần chính. Phần thứ nhất là các interface repository, định nghĩa tập các thao tác dữ liệu mà hệ thống cần sử dụng. Phần thứ hai là các lớp cài đặt cụ thể, chịu trách nhiệm làm việc với cơ sở dữ liệu và hiện thực hóa các thao tác đã được định nghĩa trong interface.

Các service hoặc module nghiệp vụ sẽ không làm việc trực tiếp với database, mà chỉ tương tác với các repository thông qua abstraction. Nhờ đó, tầng nghiệp vụ được tách biệt khỏi công nghệ lưu trữ bên dưới. Trong nhiều hệ thống, repository còn đóng vai trò tập trung hóa các truy vấn, giúp việc quản lý logic dữ liệu trở nên nhất quán hơn.

Sự phù hợp của Repository Pattern với dự án

Hệ thống Yolo:Home quản lý nhiều loại dữ liệu khác nhau như người dùng, thiết bị, phòng, dữ liệu cảm biến và nhật ký hành động. Các dữ liệu này có mối quan hệ phức tạp và được truy cập thường xuyên từ nhiều chức năng khác nhau.

Việc áp dụng Repository Pattern giúp hệ thống xử lý dữ liệu một cách thống nhất, đồng thời tránh việc logic nghiệp vụ phải làm việc trực tiếp với database. Điều này đặc biệt quan trọng trong hệ thống IoT, nơi dữ liệu liên tục được sinh ra và cần được xử lý hiệu quả.

Lý do lựa chọn

Lý do quan trọng nhất để lựa chọn Repository Pattern là nhu cầu tách biệt business logic với data access logic. Đây là yêu cầu cốt lõi của một hệ thống cần phát triển lâu dài, bởi vì dữ liệu và nghiệp vụ thường thay đổi ở tốc độ khác nhau. Logic nghiệp vụ có thể được mở rộng hoặc điều chỉnh mà không nhất thiết kéo theo thay đổi lớn ở tầng dữ liệu, và ngược lại.

Bên cạnh đó, trong một hệ thống IoT như Yolo:Home, dữ liệu cảm biến được sinh ra liên tục, đồng thời hệ thống còn phải xử lý các truy vấn từ người dùng đối với lịch sử và thống kê. Điều này làm cho tầng dữ liệu trở thành một phần quan trọng và cần được quản lý cẩn thận. Việc sử dụng Repository Pattern giúp cô lập phần phức tạp này, tránh lan truyền chi tiết lưu trữ sang toàn bộ hệ thống.

Ngoài ra, Repository Pattern cũng được lựa chọn vì nó hỗ trợ tốt cho việc kiểm thử. Khi cần kiểm tra logic của service, có thể thay thế repository thật bằng các đối tượng giả lập mà không cần phụ thuộc vào database thật. Đây là một lợi thế đáng kể trong quá trình phát triển và bảo trì.

Ưu điểm trong bối cảnh dự án

Khi áp dụng vào Yolo:Home, Repository Pattern mang lại nhiều lợi ích. Trước hết, nó giúp hệ thống có kiến trúc phân lớp rõ ràng, trong đó dữ liệu và nghiệp vụ được tách biệt tương đối độc lập. Điều này làm cho mã nguồn dễ hiểu hơn và giảm nguy cơ các tầng trộn lẫn trách nhiệm.

Tiếp theo, mẫu này hỗ trợ rất tốt cho khả năng mở rộng. Khi hệ thống cần bổ sung thêm các loại dữ liệu mới, hoặc thay đổi cách truy vấn thông kê, nhóm phát triển có thể chỉnh sửa tại tầng repository mà không phải can thiệp sâu vào các thành phần phía trên. Ngoài ra, việc tập trung hóa logic truy vấn còn giúp việc tối ưu hiệu năng và kiểm soát dữ liệu trở nên thuận tiện hơn.

Một ưu điểm khác là tính nhất quán. Trong hệ thống có nhiều nguồn dữ liệu và nhiều chức năng khác nhau, việc có một cơ chế chung để truy cập dữ liệu sẽ làm giảm sai sót và tăng khả năng tái sử dụng.

2.1.2 Dependency Injection Pattern

Khái quát về mẫu thiết kế

Dependency Injection là một mẫu thiết kế cho phép một đối tượng nhận các dependency từ bên ngoài thay vì tự tạo ra chúng. Nói cách khác, thay vì để một thành phần tự quyết định nó sẽ sử dụng những đối tượng phụ trợ nào, các dependency sẽ được cung cấp sẵn thông qua một cơ chế trung gian. Điều này làm giảm sự phụ thuộc trực tiếp giữa các thành phần và giúp hệ thống linh hoạt hơn.

Mẫu thiết kế này thường được sử dụng trong các hệ thống nhiều tầng, nơi các service, repository và module xử lý cần tương tác với nhau nhưng vẫn phải giữ được mức độ độc lập cần thiết. Dependency Injection không chỉ là một kỹ thuật lập trình mà còn là một nguyên tắc thiết kế giúp hệ thống hướng đến abstraction và loose coupling.

Cấu trúc của Dependency Injection Pattern

Cấu trúc của Dependency Injection bao gồm ba thành phần chính. Thứ nhất là các abstraction, thường là interface hoặc các hợp đồng chung mô tả hành vi của dependency. Thứ hai là các implementation cụ thể hiện thực hóa những abstraction này. Thứ ba là một cơ chế trung gian, thường được gọi là container, chịu trách nhiệm quản lý và cung cấp implementation phù hợp đến nơi cần sử dụng.

Các thành phần nghiệp vụ không cần biết cụ thể dependency được tạo ra như thế nào, cũng không cần tự khởi tạo chúng. Chúng chỉ cần khai báo nhu cầu sử dụng, còn việc cung cấp dependency sẽ do container đảm nhiệm. Cấu trúc này giúp tách biệt trách nhiệm khởi tạo khỏi trách nhiệm sử dụng.

Sự phù hợp của Dependency Injection Pattern với dự án

Hệ thống Yolo:Home bao gồm nhiều thành phần như service, repository và các module xử lý dữ liệu, tất cả đều có mối quan hệ phụ thuộc lẫn nhau. Nếu các dependency này được khởi tạo trực tiếp trong code, hệ thống sẽ trở nên khó mở rộng và khó bảo trì.

Dependency Injection giúp giải quyết vấn đề này bằng cách cho phép các thành phần chỉ phụ thuộc vào abstraction, trong khi việc cung cấp implementation được xử lý bên ngoài.

Lý do lựa chọn

Injection là nhằm giảm coupling giữa các thành phần. Trong các hệ thống có quy mô trung bình đến lớn, coupling cao là nguyên nhân phổ biến khiến việc bảo trì trở nên khó khăn. Khi một thành phần thay đổi, các thành phần khác dễ bị ảnh hưởng dây chuyền. Dependency Injection giúp hạn chế điều này bằng cách buộc hệ thống làm việc thông qua abstraction.

Một lý do khác là khả năng hỗ trợ kiểm thử. Khi dependency được cung cấp từ bên ngoài, có thể dễ dàng thay thế dependency thật bằng mock hoặc stub trong quá trình test, từ đó kiểm tra logic của từng thành phần một cách độc lập. Điều này đặc biệt quan trọng đối với hệ thống như Yolo:Home, nơi các thành phần nghiệp vụ cần được kiểm tra tách biệt khỏi database, broker hay thiết bị thật.

Ngoài ra, Dependency Injection còn phù hợp vì hệ thống có những thành phần cần được quản lý vòng đời rõ ràng. Trong môi trường IoT, một số dịch vụ hoạt động lâu dài, một số dịch vụ gắn với từng request, còn một số khác chỉ cần tồn tại tạm thời. Cơ chế DI giúp quản lý những vòng đời này một cách có hệ thống.

Ưu điểm trong bối cảnh dự án

Khi áp dụng vào Yolo:Home, Dependency Injection giúp hệ thống trở nên linh hoạt và dễ tổ chức hơn. Các service có thể được thay đổi implementation mà không làm ảnh hưởng đến những nơi sử dụng chúng. Điều này đặc biệt hữu ích khi hệ thống cần mở rộng thêm chức năng mới hoặc thay đổi công nghệ nền tảng.

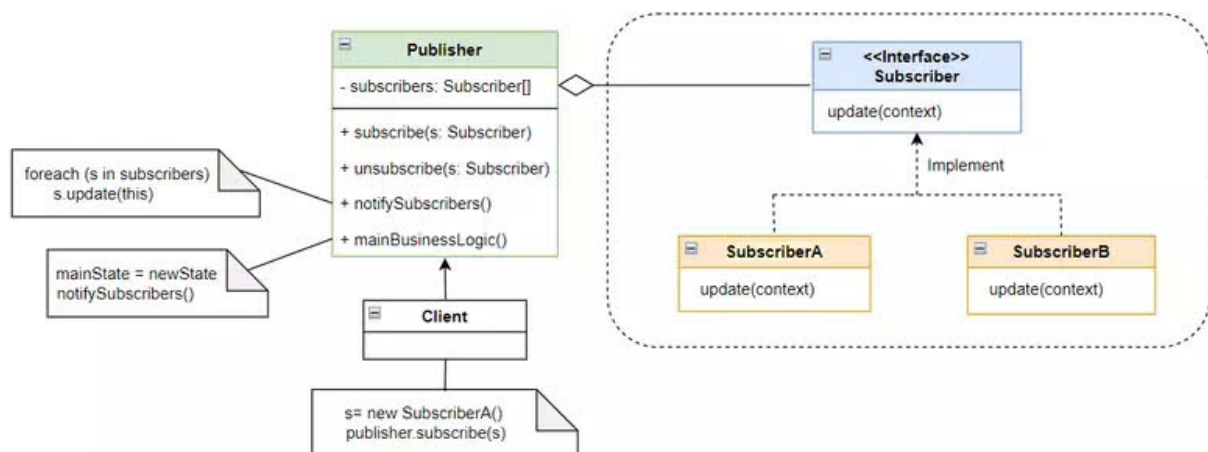
Ngoài ra, mẫu này còn giúp cấu trúc hệ thống trở nên rõ ràng hơn về mặt trách nhiệm. Các thành phần chỉ tập trung vào logic của mình, không phải kiêm luôn việc tạo và quản lý dependency. Nhờ đó, mã nguồn ngắn gọn hơn, dễ đọc hơn và ít bị lặp.

Về lâu dài, Dependency Injection tạo nền tảng tốt cho việc bảo trì và mở rộng. Khi số lượng service và module tăng lên, sự hỗ trợ của một cơ chế quản lý dependency tập trung sẽ giúp hệ thống tránh được tình trạng phụ thuộc chằng chịt, vốn là vấn đề thường gặp trong các dự án phát triển dần theo thời gian.

2.1.3 Observer Pattern

Khái quát về mẫu thiết kế

Observer là một mẫu thiết kế thuộc nhóm behavioral design pattern, được sử dụng để xây dựng mối quan hệ một-nhiều giữa các đối tượng. Trong mô hình này, một đối tượng trung tâm, thường được gọi là Subject hoặc Publisher, sẽ duy trì danh sách các đối tượng phụ thuộc, gọi là Observers hoặc Subscribers. Mỗi khi trạng thái của Subject thay đổi, toàn bộ Observer đã đăng ký sẽ được thông báo tự động để thực hiện cập nhật tương ứng.



Hình 2: kiến trúc Observer

Mẫu thiết kế này thường được sử dụng trong các hệ thống có nhu cầu phát sinh sự kiện liên tục, đặc biệt là các hệ thống giao diện người dùng, hệ thống phát tin thời gian thực, hoặc các kiến trúc dựa trên cơ chế publish–subscribe. Ưu điểm nổi bật của Observer là cho phép tách biệt thành phần phát sinh dữ liệu và thành phần nhận dữ liệu, nhờ đó giảm mức độ phụ thuộc trực tiếp giữa các module trong hệ thống.

Cấu trúc của Observer

Về mặt cấu trúc, mẫu Observer bao gồm các thành phần chính. Subject là đối tượng nắm giữ dữ liệu hoặc trạng thái cần theo dõi. Subject cung cấp các phương thức để thêm, xóa và thông báo đến các Observer. Observer là giao diện hoặc lớp trừu tượng định nghĩa phương thức cập nhật. Concrete Observer là các đối tượng cài đặt cụ thể hành vi phản ứng khi nhận được thông báo từ Subject.

Trong ngữ cảnh tổng quát, khi Subject thay đổi trạng thái, nó sẽ gọi phương thức cập nhật trên tất cả các Observer đã được đăng ký. Các Observer sau đó có thể lấy dữ liệu mới hoặc xử lý ngay dựa trên thông tin được truyền đến.

Sự phù hợp của Observer đối với dự án Yolo:Home

Yolo:Home là một hệ thống có bản chất dữ liệu thời gian thực rất rõ rệt. Dữ liệu từ cảm biến được sinh ra liên tục và cần được nhiều thành phần khác nhau sử dụng gần như đồng thời. Khi

có một bản tin nhiệt độ mới, dữ liệu này có thể đồng thời phục vụ cho việc hiển thị realtime trên dashboard, lưu trữ vào cơ sở dữ liệu, kiểm tra ngưỡng cảnh báo và ghi nhận lịch sử. Điều này tạo ra đúng bối cảnh mà Observer Pattern được thiết kế để giải quyết.

Trong hệ thống, cơ chế publish–subscribe của MQTT chính là một hiện thực thực tế rất gần với Observer. Gateway hoặc thiết bị đóng vai trò phát dữ liệu, còn các thành phần như backend và frontend đóng vai trò subscriber. Khi dữ liệu được gửi lên broker, các thành phần đã đăng ký quan tâm đến topic tương ứng sẽ được nhận thông báo và tự động xử lý. Đây là một kiến trúc rất phù hợp cho hệ thống nhà thông minh, nơi nhiều module cần phản ứng với cùng một nguồn dữ liệu mà không nên phụ thuộc trực tiếp vào nhau.

Lý do lựa chọn

Observer Pattern được lựa chọn trước hết vì hệ thống có tính chất hướng sự kiện. Dữ liệu cảm biến, trạng thái thiết bị, hành động điều khiển và cảnh báo đều là những sự kiện phát sinh theo thời gian thực. Trong bối cảnh này, một cơ chế cho phép các thành phần tự động được cập nhật khi có thay đổi là rất cần thiết.

Lý do thứ hai là một nguồn dữ liệu trong hệ thống thường không chỉ phục vụ cho một mục đích duy nhất. Một bản tin từ cảm biến nhiệt độ vừa có thể được hiển thị cho người dùng, vừa được lưu vào database, vừa được kiểm tra để kích hoạt cảnh báo. Nếu các thành phần này được nối trực tiếp với nhau theo kiểu gọi hàm tuần tự, hệ thống sẽ nhanh chóng trở nên ràng buộc chặt và khó mở rộng. Observer Pattern giúp tách những trách nhiệm này ra thành các mô-đun độc lập nhưng vẫn có thể cùng phản ứng trước một sự kiện chung.

Một lý do quan trọng khác là tính phù hợp tự nhiên với công nghệ MQTT mà hệ thống đang sử dụng. Publish–subscribe vốn là một mô hình hiện thực rất điển hình của Observer. Do đó, việc lựa chọn Observer không chỉ đúng về mặt thiết kế phần mềm mà còn thống nhất với hạ tầng truyền thông của dự án.

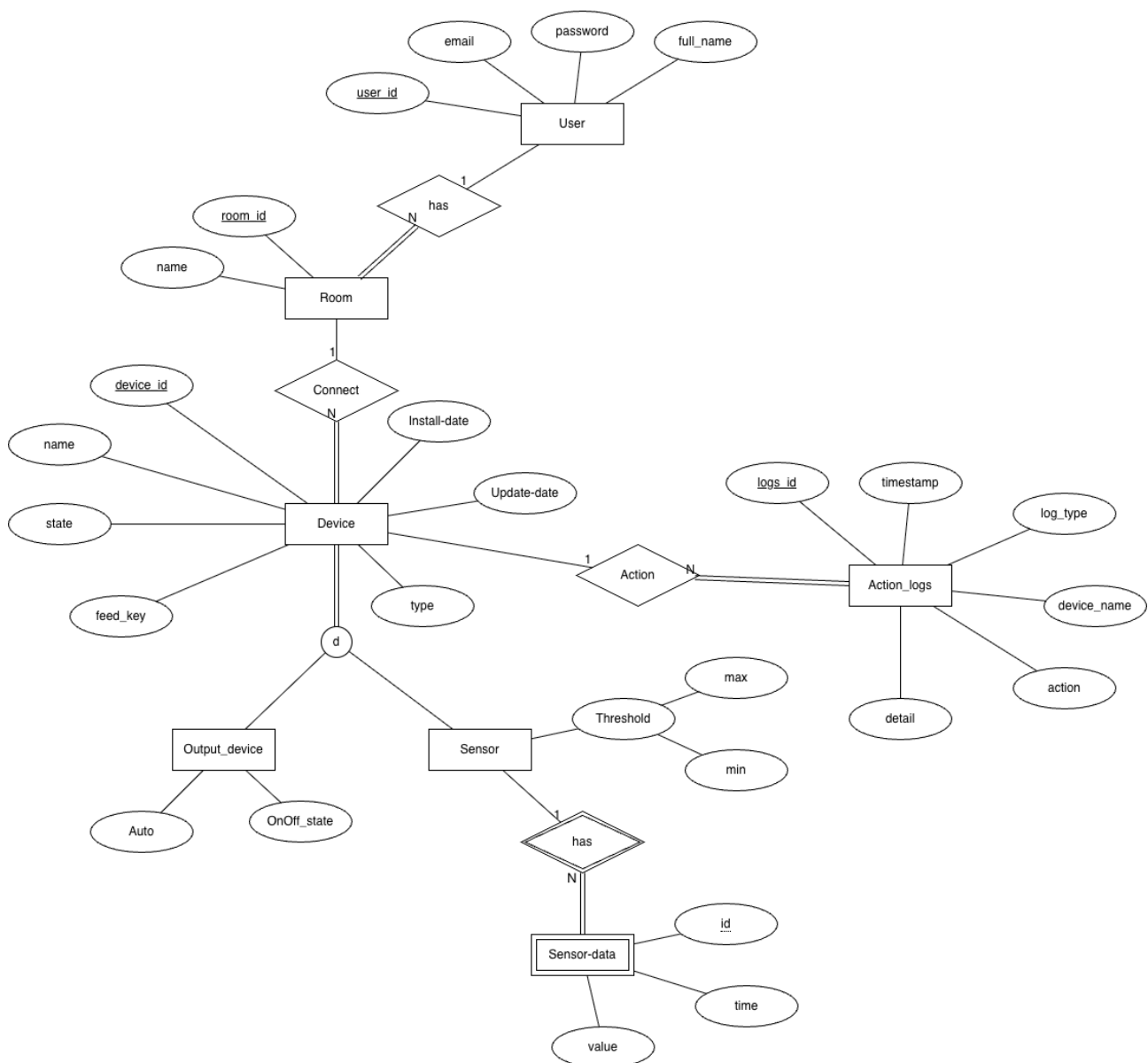
Ưu điểm trong bối cảnh dự án

Khi áp dụng vào Yolo:Home, Observer mang lại nhiều lợi ích rõ rệt. Trước hết, nó tạo ra cơ chế cập nhật dữ liệu tự động, từ đó đảm bảo dashboard và các module xử lý luôn đồng bộ với

trạng thái mới nhất của hệ thống. Tiếp theo, nó giúp phân tách rõ ràng giữa nguồn phát dữ liệu và các thành phần tiêu thụ dữ liệu, nhờ đó kiến trúc tổng thể trở nên linh hoạt và sạch hơn.

Ngoài ra, Observer còn hỗ trợ tốt cho việc mở rộng chức năng trong tương lai. Ví dụ, nếu hệ thống cần bổ sung chức năng phân tích dữ liệu bằng trí tuệ nhân tạo hoặc gửi thông báo qua ứng dụng di động, nhóm có thể tích hợp thêm observer mới mà không phải thay đổi luồng dữ liệu gốc. Điều này giúp bảo toàn kiến trúc hiện tại và giảm chi phí phát triển sau này.

2.2 Sơ đồ thực thể liên kết mở rộng(EERD)



Hình 3: Sơ đồ EERD của hệ thống

EERD của hệ thống Yolo:Home được thiết kế nhằm mô hình hóa cấu trúc dữ liệu phục vụ cho việc quản lý người dùng, thiết bị IoT, dữ liệu cảm biến và nhật ký hoạt động. Sơ đồ thể hiện các thực thể chính, thuộc tính và mối quan hệ giữa chúng, đồng thời sử dụng cơ chế chuyên biệt hóa (specialization) để phân loại thiết bị.

Thực thể User

Thực thể User lưu trữ thông tin tài khoản người dùng trong hệ thống. Các thuộc tính bao gồm `user_id` (khóa chính), `email`, `password` và `full_name`. Mỗi người dùng có thể quản lý nhiều phòng khác nhau trong hệ thống nhà thông minh.

Thực thể Room

Thực thể Room biểu diễn các phòng trong ngôi nhà. Các thuộc tính gồm `room_id` (khóa chính) và `name`. Mỗi phòng thuộc về một người dùng, thể hiện thông qua mối quan hệ “has” giữa User và Room với bản chất một-nhiều (1:N), nghĩa là một người dùng có thể sở hữu nhiều phòng.

Thực thể Device

Thực thể Device đại diện cho các thiết bị IoT trong hệ thống. Các thuộc tính gồm `device_id` (khóa chính), `name`, `state`, `type`, `feed_key`, `install_date` và `update_date`. Mỗi thiết bị được gán vào một phòng thông qua mối quan hệ “Connect” giữa Room và Device, cũng theo kiểu một-nhiều (một phòng có nhiều thiết bị). Trường `feed_key` là định danh duy nhất để Backend và Adafruit ”nói chuyện” với nhau, hoàn toàn độc lập với tên hiển thị của thiết bị (`device_name`).

Chuyên biệt hóa thiết bị

Thực thể Device được chuyên biệt hóa thành hai loại con:

- `Output_device`: đại diện cho các thiết bị chấp hành như relay, servo. Các thuộc tính bổ sung bao gồm `Auto` (chế độ tự động, dùng để xác định thiết bị đang được điều khiển bởi chế độ manual hay auto(0/1)) và `OnOff_state` (trạng thái bật/tắt).

-
- **Sensor**: đại diện cho các thiết bị cảm biến. Thực thể này liên quan đến dữ liệu cảm biến và ngưỡng cảnh báo.

Việc phân tách này giúp hệ thống dễ dàng mở rộng và quản lý các loại thiết bị khác nhau với các thuộc tính riêng biệt.

Thực thể Sensor-data

Thực thể Sensor-data lưu trữ dữ liệu đo được từ các cảm biến theo thời gian. Các thuộc tính gồm id (khóa chính), time và value. Mỗi quan hệ “has” giữa Sensor và Sensor-data thể hiện rằng một cảm biến có thể sinh ra nhiều bản ghi dữ liệu (quan hệ 1:N).

Thực thể Threshold

Thực thể Threshold được sử dụng để lưu các giá trị ngưỡng phục vụ cho việc cảnh báo. Các thuộc tính gồm min và max. Ngưỡng này được áp dụng cho các cảm biến để xác định khi nào cần kích hoạt cảnh báo.

Thực thể Action_logs

Thực thể Action_logs ghi lại các hành động điều khiển thiết bị trong hệ thống. Các thuộc tính gồm logs_id (khóa chính), timestamp, log_type, device_name, action và detail.

Mỗi quan hệ “Action” liên kết giữa Device và Action_logs cho phép hệ thống theo dõi toàn bộ lịch sử tương tác với thiết bị, phục vụ cho việc kiểm tra và phân tích.

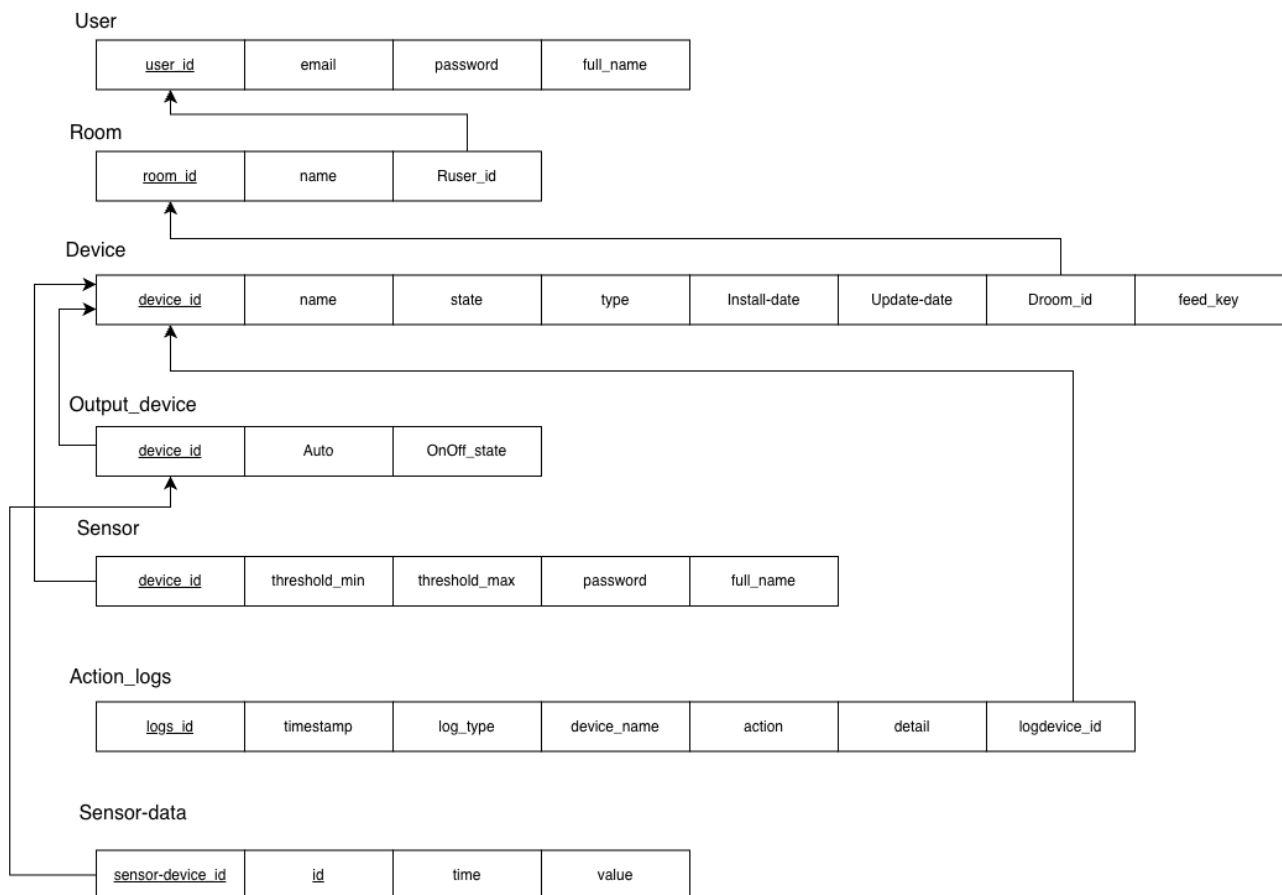
Các mối quan hệ chính

Các mối quan hệ trong hệ thống bao gồm:

- **User – Room (has)**: một người dùng có nhiều phòng.
- **Room – Device (Connect)**: một phòng có nhiều thiết bị.
- **Device – Action_logs (Action)**: một thiết bị có nhiều bản ghi hành động.
- **Sensor – Sensor-data (has)**: một cảm biến tạo ra nhiều dữ liệu theo thời gian.

Các mối quan hệ này giúp đảm bảo tính liên kết giữa các thành phần dữ liệu và hỗ trợ việc truy vấn thông tin một cách hiệu quả.

2.3 Sơ đồ ánh xạ thực thể sang mô hình quan hệ



Hình 4: Sơ đồ quan hệ của hệ thống

Dựa trên EERD đã xây dựng, hệ thống Yolo:Home được ánh xạ sang mô hình cơ sở dữ liệu quan hệ nhằm phục vụ cho việc lưu trữ và truy vấn dữ liệu. Quá trình chuyển đổi tuân theo các nguyên tắc chuẩn, trong đó mỗi thực thể được ánh xạ thành một bảng, các thuộc tính trở thành các cột và các mối quan hệ được biểu diễn thông qua khóa ngoại.

Bảng User

Bảng User được xây dựng để lưu trữ thông tin tài khoản người dùng. Các thuộc tính bao gồm user_id, email, password và full_name. Trong đó, user_id đóng vai trò là khóa chính (Primary Key), đảm bảo mỗi người dùng được định danh duy nhất trong hệ thống.

Bảng Room

Bảng Room lưu trữ thông tin các phòng trong hệ thống. Các thuộc tính gồm room_id, name và Ruser_id. Trong đó, room_id là khóa chính, còn Ruser_id là khóa ngoại (Foreign Key) tham chiếu đến user_id trong bảng User.

Mối quan hệ một-nhiều giữa User và Room được hiện thực hóa thông qua khóa ngoại này, cho phép một người dùng sở hữu nhiều phòng.

Bảng Device

Bảng Device đại diện cho các thiết bị IoT. Các thuộc tính gồm device_id, name, state, type, feed_key, install_date, update_date và Droom_id.

Trong đó:

- device_id là khóa chính
- Droom_id là khóa ngoại tham chiếu đến room_id trong bảng Room

Mối quan hệ giữa Room và Device được thể hiện qua khóa ngoại này, cho phép mỗi phòng chứa nhiều thiết bị.

Phân cấp thiết bị (Specialization Mapping)

Thực thể Device được chuyên biệt hóa thành hai bảng con:

Bảng Output_device

Bảng Output_device chứa các thiết bị chấp hành. Các thuộc tính gồm:

- device_id (khóa chính, đồng thời là khóa ngoại tham chiếu Device)
- Auto
- OnOff_state

Việc sử dụng device_id làm khóa chính giúp đảm bảo mỗi bản ghi trong bảng này tương ứng với một thiết bị trong bảng Device.

Bảng Sensor

Bảng Sensor lưu thông tin các thiết bị cảm biến. Các thuộc tính gồm:

-
- device_id (khóa chính, đồng thời là khóa ngoại)
 - threshold_min
 - threshold_max
 - password
 - full_name

Bảng này kế thừa từ bảng Device thông qua khóa ngoại device_id.

Bảng Action_logs

Bảng Action_logs được sử dụng để ghi lại các hành động điều khiển thiết bị. Các thuộc tính gồm:

- logs_id (khóa chính)
- timestamp
- log_type
- device_name
- action
- detail
- logdevice_id (khóa ngoại)

Thuộc tính logdevice_id tham chiếu đến device_id trong bảng Device, thể hiện mối quan hệ giữa thiết bị và các hành động đã thực hiện.

Bảng Sensor_data

Bảng Sensor_data lưu trữ dữ liệu cảm biến theo thời gian. Các thuộc tính gồm:

- id (khóa chính)
- sensor-device_id (khóa ngoại)
- time
- value

Khóa ngoại sensor-device_id tham chiếu đến device_id trong bảng Sensor, cho phép mỗi cảm biến có nhiều bản ghi dữ liệu theo thời gian, thể hiện mối quan hệ một-nhiều.